Coherent Control of Delayed Nonholonomic Articulated Systems: Resolving Reversing Instability in Tractor-Trailer Reinforcement Learning Policies

Kinematic Instability and the Catastrophe of Actuation Delay

Articulated vehicle systems, such as tractor-trailers, represent a class of highly non-linear, underactuated, and open-loop unstable dynamical plants when operated in reverse.[1] While the mechanical coupling of a tractor-trailer exhibits self-stabilizing characteristics during forward travel, backward motion completely reverses these stability margins.[1] In reverse, the zero dynamics associated with the articulation joint become unstable, behaving analogously to an active, nonholonomic inverted pendulum on a cart.[2]

To mathematically characterize this instability, the rate of change of the hitch angle (or articulation angle) θ_a between the tractor and the trailer can be modeled using a standard kinematic bicycle formulation [5]:

$$\dot{\theta}_a = \frac{v}{L_2}\sin(\theta_a) - \frac{v}{L_1}\tan(\delta)$$

Within this kinematic model, v denotes the backward velocity of the tractor (where $v > 0$ during reverse maneuvers), L_1 represents the tractor wheelbase, L_2 represents the trailer hitch length (specifically the distance from the hitch point to the trailer axle), and δ constitutes the steering actuation input of the tractor's steerable front wheels.[5]

The primary driver of reversing instability is the positive feedback loop governed by the term $\frac{v}{L_2}\sin(\theta_a)$.[3] Any non-zero perturbation in the articulation angle θ_a causes a self-reinforcing lateral rotation that pushes the trailer further from the tractor's longitudinal axis.[3] If left uncorrected, this rotation quickly culminates in a catastrophic "jackknife" state where the physical limitations of the joint are exceeded and control becomes mathematically impossible.[1]

The physical mechanism of this instability is rooted in the force characteristics at the coupling joint.[1] When a trailer is pulled forward, the interaction forces F_{h1} and F_{h2} pull the tractor, generating moments that actively suppress tractor yaw.[1] Conversely, when reversing, the trailer longitudinally pushes the tractor forward.[1] This shift in force direction causes the resulting hitch moments M_{h1} and M_{h2} to actively promote tractor yaw, exponentially increasing the probability of jackknife instability.[1]
To prevent this, the controller must turn the tractor more rapidly than the trailer is rotating.[3] This requires continuous, low-amplitude, and high-frequency steering corrections.[3]

When steering actuation delay τ is introduced to the simulation environment, the control input actually executed by the tractor is lagged:

$$\dot{\theta}_a(t) = \frac{v}{L_2}\sin(\theta_a(t)) - \frac{v}{L_1}\tan(\delta(t-\tau))$$

In a delay-free setting ($\tau = 0$), a reinforcement learning agent can instantly observe a deviation in θ_a and apply an immediate, precise steering correction $\delta(t)$ to neutralize the destabilizing moment.[3] However, when $\tau > 0$, even a tiny latency means that the counter-steering command is executed too late, while the positive feedback term $\frac{v}{L_2}\sin(\theta_a(t))$ continues to grow unhindered in real-time.[3]
By the time the command $\delta(t-\tau)$ physically registers at the steerable wheels, the true articulation angle has drifted past the point where the commanded input is effective.[3] This latency introduces a severe phase lag in the feedback loop.[9] The reinforcement learning policy, operating under the assumption of immediate action execution, observes a worsening error

and responds with increasingly aggressive counter-steering commands.[3] This mismatch generates severe steering chattering, destabilizing limit cycles, and rapid jackknifing.[3]
From a decision-theoretic perspective, the addition of actuation delay violates the core Markov Decision Process (MDP) assumption underpinning standard reinforcement learning algorithms.[8] In a classic MDP, the transition to state s_{t+1} depends solely on the current state s_t and the selected action a_t.[11] With actuation latency, the state transition depends on the history of pending actions $[a_{t-\tau}, \dots, a_{t-1}]$ that have been dispatched but not yet realized in the simulation.[13] This transitions the system into a delayed Markov Decision Process, which behaves as a highly challenging partially observable Markov decision process (POMDP).[14]
Furthermore, if critical states such as the instantaneous wheel steering angles are not explicitly included in the policy's observation space, severe state-aliasing occurs.[16] The policy cannot distinguish whether a change in the trailer's trajectory is due to external noise or a delayed action currently propagating through the actuator pipeline, preventing stable policy convergence.[8]

Delay-Aware State Representation and Sequence Encoding

To recover the Markov property within delayed environments, the policy's input space must be reformulated to encompass temporal sequences.[13] The Delay-Aware Markov Decision Process (DA-MDP) framework reconstructs a standard MDP in a mathematically lifted state space by appending a history of past observations and control actions directly to the current observation vector.[13]

For a system experiencing an actuation or observation delay of τ steps, the augmented state vector $\tilde{s}_t$ is defined as [13]:

$$\tilde{s}_t = [s_t, s_{t-1}, \dots, s_{t-\tau_o}, a_{t-1}, a_{t-2}, \dots, a_{t-\tau_o}]$$

This formulation explicitly accounts for the actions currently in transit, providing the policy with the full causal history required to estimate the future evolution of the vehicle.[13] However, simply passing a raw, flattened action history buffer to a standard feedforward multi-layer perceptron (MLP) significantly increases the dimensionality of the input space.[13] This dimensionality expansion degrades sample efficiency and destabilizes value-function estimation, especially as the delay length increases.[8]
To mitigate this, a compressed *Belief Representation* of the action history can be utilized to map the augmented buffer into a lower-dimensional latent space.[18] This approach reduces computational complexity while maintaining the necessary temporal observability.[18]
An advanced architecture designed to process this temporal sequence is the *Delay-Aware Reinforcement Learning for On-ramp Merging* (DAROM) framework, which can be directly

adapted to the tractor-trailer reversing task.[19] Under the DAROM paradigm, the policy is conditioned on a *Delayed Observation* $o_t = s_{t-\omega_t}$ (where ω_t is the current delay magnitude), a *Masked Action History* u_t, and a scalar *Delay Indicator* ω_t.[19] To strictly assist the neural network in causal inference, a temporal mask is applied to the action buffer:

$$a_{t-k} = 0 \quad \text{for} \quad k > \omega_t$$

This masking operation zeroes out actions that have already contributed to the transition from the historical state to the current delayed observation o_t, isolating only the pending actions that will shape the future trajectory of the vehicle.[19]
The choice of sequence-encoding architecture is critical for parsing these history-augmented states, as detailed in the comparison below:

Sequence Architecture	Input Processing Mechanism	Temporal Representation Quality	Computational & Sample Efficiency
Multi-Layer Perceptron (MLP) [20]	Concatenates and flattens the delayed state o_t and the action history buffer u_t into a single static vector.[13]	Low; struggles to capture the sequential dependencies and causal ordering of actions under variable latencies.[13]	High efficiency per gradient step; extremely poor sample efficiency and poor convergence in complex environments.[13]
Gated Recurrent Unit (GRU) / LSTM [20]	Processes the action history sequence autoregressively to maintain a continuous, hidden temporal belief state.[9]	High; exceptionally robust at capturing the long-term physical dependencies of the unstable articulation joint.[9]	Medium efficiency; requires backpropagation through time (BPTT) but significantly stabilizes policy learning.[9]
Transformer-Based Encoder [20]	Utilizes self-attention heads to dynamically weigh historical	Very High; excels at identifying non-linear temporal correlations in	Low efficiency; requires substantial training data and has higher

	actions relative to the delayed state observation.[20]	stochastic, high-variance delay regimes.[20]	computational overhead during policy inference.[13]

Ablation studies conducted within delay-resolved control environments demonstrate that omitting key components of this augmented state vector severely compromises performance.[20] The empirical impact of different observation configurations is outlined in the following table:

Observation Configuration	**Action Buffer (ut)**	**Delay Indicator (ωt)**	**Control Stability in Stochastic Latency Regimes**
Delayed State Only [20]	Excluded	Excluded	**Fails completely**; high-frequency chattering and immediate jackknifing due to zero temporal awareness.[8]
Delayed State with Delay Magnitude [20]	Excluded	Included	**Fails to stabilize**; the agent understands the age of the observation but cannot project its unexecuted steering actions.[20]
Delayed State with Action Buffer [20]	Included	Excluded	**Marginally stable**; acceptable under constant delays but degrades rapidly under stochastic or time-varying latency.[20]
Full Delay-Aware Encoder [20]	Included (Masked)	Included	**Highly stable**; recovers the true current state representation,

			enabling smooth, robust reversing maneuvers.[20]

Predictor-Based and Model-Aware Control Architectures

Rather than training a model-free policy to navigate a high-dimensional, history-augmented state space, predictor-based frameworks attempt to reconstruct the current, instantaneous state of the vehicle.[9] This allows the reinforcement learning agent to operate in a virtual, delay-free environment, preserving its original performance.[23]

One approach uses a predictive model P to approximate the delayed observation.[23] The transition dynamics are learned via a neural network, allowing the system to project the state forward in time using the pending actions in the queue [18]:

$$\hat{s}_{t-i} = P(\hat{s}_{t-i-1}, a_{t-i-\tau}) \quad \text{for} \quad i = \tau - 1, \dots, 0$$

To provide stable, robust convergence, the neural network can be trained utilizing a Huber loss to minimize the prediction error [18]:

$$L_{\text{Huber}} = \begin{cases} \frac{1}{2}(s_i - \hat{s}_i)^2, & \text{if } |s_i - \hat{s}_i| < \delta \\ \delta(|s_i - \hat{s}_i| - \frac{1}{2}\delta), & \text{otherwise} \end{cases}$$

This loss formulation limits the influence of outliers and sensor noise, which is critical when predicting the highly sensitive, non-linear dynamics of an articulating hitch.[2]

Alternatively, *Model-Based Reinforcement Learning* (MBRL) frameworks, such as *CausalDreamer* or *Dreamer-V3*, learn a world model directly in a latent space.[15] CausalDreamer models the environment as multiple field nodes with independent semantics, utilizing a message-passing mechanism to characterize dynamic causal dependencies under random delays.[17] By simulating future trajectories within the latent space of the world model, the policy can plan and train entirely within an "imagined" delay-free horizon, bypassing the physical latency of the simulator.[15]

Another approach is the *Delay-resilient Encoder-Enhanced RL* (DEER) framework.[24] DEER pre-trains a sequence-to-sequence encoder on offline, delay-free trajectories to reconstruct the environment's latent state.[24] During online deployment, this encoder maps the incoming delayed observations and recent action histories into a compressed, fixed-length context vector, allowing a standard, delay-free actor-critic policy (such as Soft Actor-Critic) to operate effectively without modifying its core objective.[24]

For systems where a kinematic model of the plant is available, the **Smith Predictor** can be integrated with Deep Reinforcement Learning (SP-DRL).[25] The Smith Predictor mitigates the

impact of dead-time by utilizing a primary, delay-free model of the plant to anticipate the system's output, while a delayed model path compares the predicted delayed output against the true delayed feedback.[25] The DRL policy is trained on the output of the delay-free primary path, allowing for straightforward controller design.[25]

To analyze the stability limits of the Smith Predictor under latency, let $G_p(s) = P_0(s)e^{-\zeta s}$ represent the transfer function of the plant with a dead-time ζ.[25] Using a first-order Padé approximation for the delay term, $e^{-\zeta s} \approx 1 - \zeta s$, the closed-loop system stability boundaries can be evaluated via the Routh-Hurwitz criterion, yielding [25]:

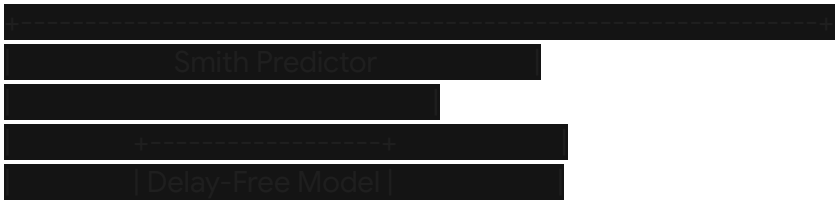
$$-1 < k < \frac{\tau}{\zeta}$$

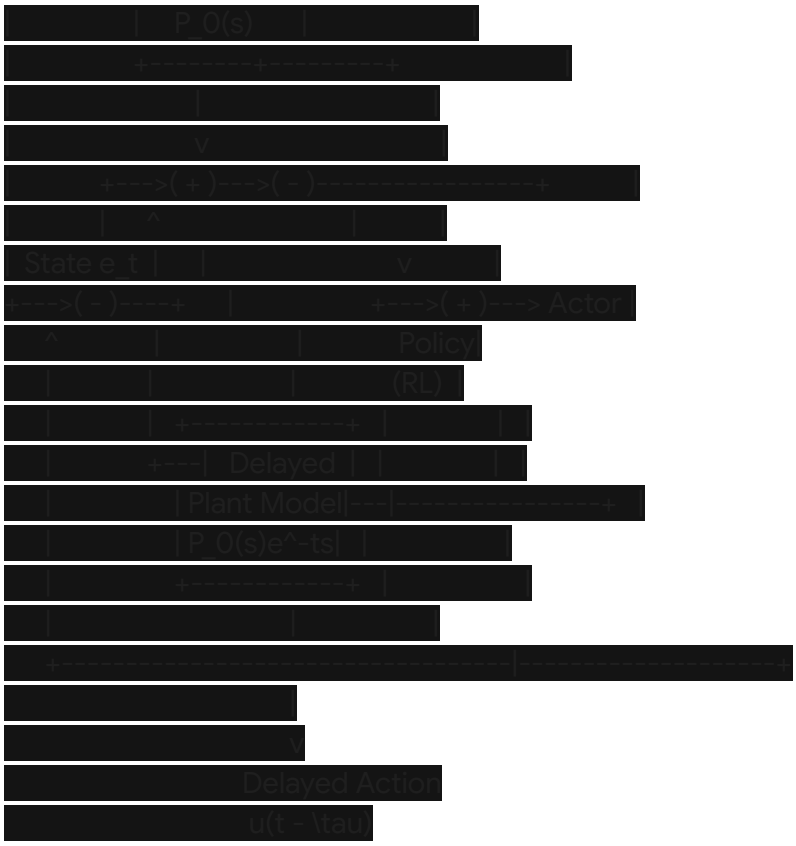
where k represents the controller gain and τ is the system time constant.[25] When the physical system experiences stochastic, time-varying delays or significant model-plant mismatches (such as tire slip and variable ground friction), the predictive performance of the standard Smith Predictor deteriorates.[9]

To address these limitations, a *Composite Actor-Critic* reinforcement learning scheme can be built.[26] This framework optimizes a tracking error $e(t) = y_o(t) - y_d(t)$ by minimizing a quadratic cost function [26]:

$$J = \int_0^\infty \left(e(t)^T Q_e e(t) + u_c(t)^T R u_c(t) \right) dt$$

where Q_e and R are state error and control weighting matrices, respectively.[26] By augmenting the system states with the tracking error and utilizing concurrent learning with data collected from the known system model, the actor-critic scheme approximates the optimal control policy $u_c(t)$ in real-time, maintaining stability despite delay mismatch.[26]





In scenarios characterized by high-variance stochastic delays, standard *Single-Step State Predictors* (SBSP) suffer from discontinuous state estimates.[9] SBSP updates its prediction only upon the arrival of a new delayed packet; between arrivals, the output is held constant, causing the state estimate to jump discontinuously when a new measurement is processed.[9]
To resolve this, a hybrid **LSTM Estimator-Residual Control** framework can be deployed.[9] An LSTM-based state estimator reconstructs continuous, smooth state estimates from the delayed feedback stream.[9] This continuous estimate is utilized by a low-level Proportional-Derivative (PD) controller to maintain basic closed-loop stability.[9]
Simultaneously, a residual deep reinforcement learning policy is trained to output corrective steering adjustments, compensating for two specific error sources: tracking deviations caused by predictor errors under stochastic delay, and unmodeled non-linear dynamics that the PD gains cannot suppress.[9] This division of labor ensures high-frequency control continuity while leveraging the non-linear compensation capabilities of DRL.[9]

Credit Assignment and Multi-Step Return Bootstrapping

Actuation delay decouples the immediate execution of a steering command from its downstream physical consequences, complicating the credit assignment problem.[17] In standard Temporal Difference (TD) learning, the value function is updated using single-step bootstrapping [23]:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

When a steering delay is present, the immediate reward r_t and the next state s_{t+1} do not reflect the influence of action a_t, as that action is still propagating through the actuator pipeline.[23] Consequently, single-step TD learning assigns the reward to the incorrect action, leading to slow convergence or total policy divergence.[13]

To align the reward signal with the causal action, the policy's update step can be modified to use N**-Step Bootstrapping**.[32] By delaying the value update until N transitions have been collected, the Q-value estimate is conditioned on the actual, realized physical effects of the action [32]:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[\sum_{i=0}^{N-1} \gamma^i r_{t+i} + \gamma^N \max_a Q(s_{t+N}, a) - Q(s_t, a_t) \right]$$

This temporal bootstrapping mechanism ensures that the credit for stabilizing or destabilizing the hitch angle is correctly backpropagated to the steering command that initiated the maneuver.[23]

Furthermore, multi-step return formulations, such as the n-step return and the λ-return, have been mathematically proven to maintain convergence to the optimal policy of a standard MDP under long propagation delays without requiring explicit delay modeling.[33] By extending the temporal horizon of the return estimation to encompass the maximum delay boundary of the system, the learning process remains stable.[33]

A specific implementation of this is the **Credit Horizon-Limited λ-Return (CHILL-Return)** mechanism.[33] CHILL-Return limits the return estimation to a credit horizon matched to the maximum delay bound of the simulator, preventing asynchronous, delayed rewards from injecting high-variance noise into the value gradient.[33]

To complement these temporal modifications, integrating classical stabilization mechanisms can prevent the agent from entering unrecoverable regions of the state space.[1] A **Sliding Mode Controller (SMC)** combined with a jackknife prevention override can enforce strict mathematical boundaries on the steering rate, ensuring safe reverse driving.[2]

Additionally, a hybrid **Neuro-Fuzzy Controller** (e.g., combining the Deep Deterministic Policy Gradient (DDPG) algorithm with a fuzzy logic supervisor) can be implemented.[6] The neuro-fuzzy controller uses the DRL agent to output nominal steering commands based on the tracking error [6]:

$$\delta_0 = \mu(s - s_{goal}) + \delta_{steady\_state}$$

This nominal command is then processed by a fuzzy rule-base that monitors the hitch angle θ_a and its rate of change.[6] If the system approaches a jackknife state, the fuzzy supervisor overrides the policy, applying counter-steering to stabilize the hitch.[6] This allows the reinforcement learning agent to learn the path-following task while the fuzzy supervisor guarantees safety, accelerating convergence.[6]

Policy Regularization, Action Smoothing, and Deployment Constraints

A major challenge when deploying reinforcement learning policies on physical steerable actuators is high-frequency action oscillation.[36] Because the policy is highly sensitive to minor state perturbations, the phase lag caused by the actuation delay often leads the agent to output rapid, maximum-amplitude steering corrections.[9] This steering chattering reduces the lifespan of physical components and can trigger safety-critical failures during deployment.[36] To enforce a smooth steering profile, several policy regularization and action smoothing techniques can be integrated directly into the reinforcement learning loss function.[36]

Conditioning for Action Policy Smoothness (CAPS)

The CAPS framework introduces temporal and spatial smoothness penalties directly to the actor's loss function, regulating output variations without altering the network's inference-time architecture.[36] The total regularization loss is formulated as [36]:

$$L_{\text{CAPS}} = \lambda_t \mathbb{E}_{s_t \sim \mathcal{D}} \left[\|\pi_\theta(s_t) - \pi_\theta(s_{t-1})\|^2 \right] + \lambda_s \mathbb{E}_{s_t \sim \mathcal{D}, \bar{s} \sim \mathcal{N}(s_t)} \left[\|\pi_\theta(s_t) - \pi_\theta(\bar{s})\|^2 \right]$$

where the temporal term penalizes steering velocity between adjacent timesteps, and the spatial term minimizes output variance over a local neighborhood $\mathcal{N}(s_t)$ of similar states.[36]

Action Smoothing by Aligning Actions with Predictions from Preceding States (ASAP)

While CAPS relies on synthetic spatial perturbations that may not reflect the true state distribution, the ASAP framework defines similar states using the environment's transition distribution $P(\cdot|s_{t-1})$.[36] ASAP enforces local Lipschitz continuity by aligning the current action with the expected policy output under this transition-induced neighborhood.[36] To suppress high-frequency steering chattering, ASAP also penalizes the second-order differences of actions [36]:

$$L_{\text{ASAP\_second\_order}} = \mathbb{E}\left[\|a_t - 2a_{t-1} + a_{t-2}\|^2\right]$$

This formulation restricts the steering acceleration, forcing the policy to output smooth, continuous trajectories.[36]

Action Jacobian Penalty

To prevent the policy from developing extreme sensitivity to minor state perturbations, the Action Jacobian Penalty directly limits the gradient of the action with respect to the state using auto-differentiation [37]:

$$L_{\text{Jacobian}} = \mathbb{E}_{s\sim\mathcal{D}}\left[\left\|\frac{\partial \pi_\theta(s)}{\partial s}\right\|_F^2\right]$$

By penalizing the Frobenius norm of the policy's Jacobian, the network is restricted from executing sharp, sudden steering changes in response to small deviations in the observed hitch angle.[37] This helps eliminate unachievable high-frequency signals and reduces the policy's control bandwidth.[37]

The trade-offs associated with these smoothing techniques are summarized in the following table:

Regularization Method	Mathematical Objective	Computational Overhead	Physical Phase Lag	Performance Impact on Control
CAPS [36]	Minimizes first-order temporal differences and spatial variance.[36]	Low; requires simple sequence and neighborhood sampling.[36]	Zero; applied entirely during training as a loss penalty.[36]	High; may lead to overly conservative steering if the spatial neighborhood is hand-tuned poorly.[36]
ASAP [36]	Minimizes second-order differences and aligns actions with transition-induced states.[36]	Medium; requires sampling from the transition kernel.[38]	Zero; enforces smoothness through mathematical policy constraints.[36]	Very High; maintains high task performance while completely eliminating

				high-frequency chattering.[36]
Action Jacobian Penalty [37]	Penalizes the Frobenius norm of the policy's state-action gradient.[37]	High; requires backpropagating second-order derivatives through the network.[36]	Zero; restricts policy sensitivity during the offline optimization phase.[37]	High; generates smooth steering signals without requiring task-specific parameter tuning.[37]
ZAPS-DA [10]	Imitates zero-phase filtered targets via a decoupled actor network.[10]	Low during inference; requires training a dual-actor architecture.[10]	Zero; achieves causal distillation of a non-causal target filter.[10]	Outstanding; achieves near-perfect action smoothing without introducing any phase lag.[10]

Applying a post-hoc filter (such as a low-pass or exponential filter) to the steering output at deployment is a common baseline.[10] However, in delayed continuous control tasks, physical filters introduce a **phase lag** that compounds the existing steering delay, often driving the unstable system into immediate self-reinforcing oscillations.[10]

The **Zero-Phase Action Policy Smoothing with Decoupled Actor (ZAPS-DA)** framework avoids this issue by training a dual-actor system.[10] The main actor is trained using standard reinforcement learning losses.[10] During training, the transitions stored in the replay buffer are smoothed post-hoc using a non-causal, zero-phase filter (which eliminates phase lag but requires looking forward in time, making it undeployable in real-time).[10] A separate *decoupled actor* is then trained via supervised learning to map the current state directly to these smooth, zero-phase filtered targets.[10] At deployment, only this decoupled actor is run, outputting a smooth steering command with zero phase lag.[10]

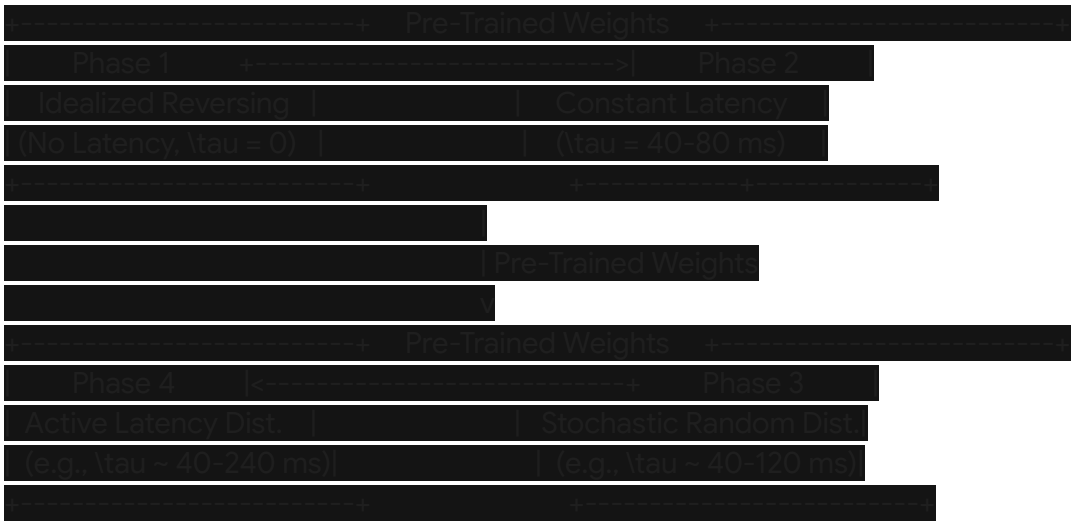
Robustness Verification via Latency Randomization and Curriculum Learning

If a reinforcement learning policy is trained in a simulator with a constant steering delay (e.g., exactly 80 ms), it will typically learn a narrow temporal correction strategy.[11] However, physical systems and networked simulation channels are characterized by stochastic, time-varying latencies.[9] When exposed to a delay that deviates from its training setting, a policy trained on a

constant delay often fails due to a lack of generalized timing representation.[9]
To resolve this, **Domain Randomization (DR)** must be applied to the simulator's latency parameters.[39] During training, the steering actuation delay τ is sampled dynamically at the start of each episode from a uniform or Gaussian distribution [9]:

$$\tau \sim \mathcal{U}(\tau_{\min}, \tau_{\max})$$

Randomizing the physical latency forces the policy to learn a generalized, robust control strategy.[39] Rather than timing its counter-steering to a specific millisecond boundary, the policy learns to prioritize gradual, predictive steering inputs that stabilize the trailer's hitch angle across a wide range of delay regimes.[36]
However, introducing heavy latency randomization from the start of training often prevents the agent from learning.[39] The combination of unstable reversing kinematics, jackknife risks, and time-varying delay makes the policy's exploration space highly challenging, leading to poor learning signals.[3]
To address this, **Continual Domain Randomization (CDR)** can be deployed.[39] CDR structures the learning process into sequential tasks, progressively introducing physical complexities:



This sequential curriculum ensures steady policy convergence.[39] The agent first masters the coordinate transformations and basic hitch-angle stabilization rules under ideal conditions.[39] It then learns to adapt its steering behavior to handle constant delays before ultimately developing robustness against high-variance, stochastic latency regimes.[39]

Alternatively, **Active Domain Randomization (ADR)** frames parameter selection as an active learning problem.[41] ADR maintains two policies: a *task policy* $\pi_\theta(a|s)$ trained to perform the reversing task, and a *sampling policy* $\pi_\omega(\phi)$ trained to select challenging environment parameters.[41]
By using a reward signal derived from a learned discriminator, the sampling policy is trained to generate environments where the agent's behavior deviates significantly from its behavior in a reference (easy) environment.[41] This ensures the agent focuses its training on edge-case latencies and high-friction boundaries, resulting in a robust policy suitable for real-world deployment.[8]

Technical Recommendations for Policy Architecture Modification

To resolve the failure of the tractor-trailer reverse driving policy under steering actuation delay, several modifications should be integrated into the reinforcement learning pipeline.

State Augmentation with Sequence Processing

The observation space must be modified to include a history of past actions and states to preserve the Markov property.[13] Rather than using a standard feedforward MLP, a Gated Recurrent Unit (GRU) should be implemented to encode this sequence.[20]
The GRU should process a sliding history window of the past K steering actions and lateral tracking errors, outputting a compressed latent vector z_t.[18] This latent vector is then concatenated with the instantaneous state vector s_t —which must include the tractor velocity, steering wheel angle, and articulation angle θ_a —before being passed to the actor-critic policy heads.[16]

Loss-Based Action Smoothing

To suppress steering chattering under delayed feedback, an action smoothness penalty must be incorporated into the actor's objective function.[36] The Action Smoothing by Aligning Actions with Predictions from Preceding States (ASAP) framework is recommended.[36]
This is achieved by adding a second-order action difference penalty directly to the policy loss, which restricts sudden changes in steering acceleration [36]:

$$L_{\text{smooth}} = \beta \mathbb{E}\left[\|a_t - 2a_{t-1} + a_{t-2}\|^2\right]$$

Setting the regularization weight $\beta \in [0.05, 0.10]$ penalizes high-frequency oscillations without restricting the policy's ability to execute necessary, low-frequency steering

maneuvers.[36]

Multi-Step Return Bootstrapping

The standard single-step temporal difference update must be replaced with an N-step bootstrapping or a Credit Horizon-Limited λ-Return (CHILL-Return) formulation.[32] This ensures that the delayed physical consequences of a counter-steering action (which may take several simulation steps to manifest as a reduction in the hitch angle θ_a) are correctly attributed to the initiating action, resolving the credit assignment problem.[23]

Safety-Constrained Hybrid Control

To prevent the agent from frequently entering unrecoverable regions during the exploration phase, a hybrid neuro-fuzzy supervisor or a sliding mode anti-jackknife override should be deployed.[2]

This safety layer monitors the instantaneous articulation angle θ_a.[2] If the joint angle exceeds a critical boundary (e.g., $|\theta_a| \geq 40^\circ$), the supervisor overrides the policy's commanded steering input and applies maximum counter-steering to stabilize the hitch.[2] This keeps the vehicle within its controllable basin of attraction, allowing the agent to collect high-value, near-equilibrium trajectories and accelerating training convergence.[2]

Progression Curriculum

A progressive training curriculum must be implemented to manage the complexity of the learning process.[39] The agent should first be trained in a delay-free simulation ($\tau = 0$) to learn the basic mechanics of reverse path-following.[39]
Once this baseline policy is established, a small constant delay should be introduced, using the pre-trained weights to initialize the network.[39] Finally, the delay parameter should be randomized using an Active Domain Randomization (ADR) scheduler, progressively widening the latency boundaries to build robust resistance to stochastic delays.[39]

Works cited

1. Research on Reverse Path Tracking Control for Hinged Unmanned Mining Truck Based on NN-SMC - MDPI, accessed June 5, 2026, https://www.mdpi.com/2075-1702/14/6/590
2. Sliding Mode Controller for Autonomous Tractor-Trailer Vehicle Reverse Path Tracking, accessed June 5, 2026, https://www.mdpi.com/2076-3417/13/21/11998
3. Article - Why Can't I Backup Straight? - towgo, accessed June 5, 2026, https://towgo.com/pages/article-why-can-t-i-backup-straight
4. MEMETIC LEARNING: A NOVEL LEARNING METHOD FOR MULTI-ROBOT

SYSTEMS - Computer Science - The University of Oklahoma, accessed June 5, 2026, https://www.cs.ou.edu/~hougen/mrs2003.pdf
5. Neural Distance-Guided Path Integral Control for Tractor-Trailer Navigation - arXiv, accessed June 5, 2026, https://arxiv.org/html/2605.09939v1
6. Backing Up Control of a Self-Driving Truck-Trailer Vehicle with Deep Reinforcement Learning and Fuzzy Logic - ResearchGate, accessed June 5, 2026, https://www.researchgate.net/publication/331202855_Backing_Up_Control_of_a_Self-Driving_Truck-Trailer_Vehicle_with_Deep_Reinforcement_Learning_and_Fuzzy_Logic
7. Master CDL Offset Backing: Pass Your Test Now | Off Set Backing Tractor Trailer, accessed June 5, 2026, https://off-set-backing-tractor-trailer.pages.dev/
8. Application and Experimental Verification of a Delay-Aware Deep Reinforcement Learning Method in End-to-End Autonomous Driving Control - Fuji Technology Press, accessed June 5, 2026, https://www.fujipress.jp/jrm/rb/robot003700051024/
9. Residual Reinforcement Learning for Robot Teleoperation under Stochastic Delays - arXiv, accessed June 5, 2026, https://arxiv.org/html/2605.15480v1
10. ZAPS-DA: Zero-Phase Action Policy Smoothing with Decoupled Actor for Continuous Control in Reinforcement Learning - Robotics Center, accessed June 5, 2026, https://www.roboticscenter.ai/research/papers/zaps-da-zero-phase-action-policy-smoothing-with-decoupled-actor-for-continuous-control-in-2605
11. Adaptive Reinforcement Learning for Real-World Systems with Delays - Diva-Portal.org, accessed June 5, 2026, https://www.diva-portal.org/smash/get/diva2:1940424/FULLTEXT01.pdf
12. Reinforcement Learning for Control Systems with Time Delays: A Comprehensive Survey, accessed June 5, 2026, https://www.researchgate.net/publication/400369827_Reinforcement_Learning_for_Control_Systems_with_Time_Delays_A_Comprehensive_Survey
13. Reinforcement Learning for Control Systems with Time Delays: A Comprehensive Survey, accessed June 5, 2026, https://arxiv.org/html/2602.00399v1
14. Reinforcement Learning from Delayed Observations via World Models - arXiv, accessed June 5, 2026, https://arxiv.org/html/2403.12309v1
15. Reinforcement Learning from Delayed Observations via World Models - arXiv, accessed June 5, 2026, https://arxiv.org/html/2403.12309v2
16. Including previous action into RL observation : r/reinforcementlearning - Reddit, accessed June 5, 2026, https://www.reddit.com/r/reinforcementlearning/comments/1jby350/including_previous_action_into_rl_observation/
17. Transferable Delay-Aware Reinforcement Learning via Implicit Causal Graph Modeling, accessed June 5, 2026, https://arxiv.org/html/2605.12312v1
18. Adaptive PD Control using Deep Reinforcement Learning for Local-Remote Teleoperation with Stochastic Time Delays - arXiv, accessed June 5, 2026, https://arxiv.org/pdf/2305.16979
19. Delay-Aware Reinforcement Learning for Highway On-Ramp Merging under

Stochastic Communication Latency - arXiv, accessed June 5, 2026, https://arxiv.org/html/2403.11852v4
20. Delay-Aware Reinforcement Learning for Highway On-Ramp Merging under Stochastic Communication Latency - arXiv, accessed June 5, 2026, https://arxiv.org/html/2403.11852v5
21. Memory-based control with recurrent neural networks - UC Berkeley Robot Learning Lab, accessed June 5, 2026, https://rll.berkeley.edu/deeprlworkshop/papers/rdpg.pdf
22. Belief-Based Offline Reinforcement Learning for Delay-Robust Policy Optimization - arXiv, accessed June 5, 2026, https://arxiv.org/html/2506.00131v2
23. How to deal with the time delay in reinforcement learning? - AI Stack Exchange, accessed June 5, 2026, https://ai.stackexchange.com/questions/25178/how-to-deal-with-the-time-delay-in-reinforcement-learning
24. Delay-Aware Reinforcement Learning with Encoder-Enhanced State Representations, accessed June 5, 2026, https://ieeexplore.ieee.org/document/11396950/
25. Investigating the Effectiveness of Reinforcement Learning in Closed-Loop Systems with Time Delays - NSF PAR, accessed June 5, 2026, https://par.nsf.gov/servlets/purl/10543317
26. Output Tracking for Uncertain Time-Delay Systems via Robust Reinforcement Learning Control - IEEE Xplore, accessed June 5, 2026, https://ieeexplore.ieee.org/iel8/10661363/10661196/10662811.pdf
27. Investigating the Effectiveness of Reinforcement Learning in Closed-Loop Systems with Time Delays - IEEE Xplore, accessed June 5, 2026, https://ieeexplore.ieee.org/document/10644470/
28. Iterative Learning Control for Smith Predictor | PDF - Scribd, accessed June 5, 2026, https://www.scribd.com/document/133008579/1-s2-0-S0167691101001426-main
29. Smith-predictor-based variable-universe fuzzy position control for hydraulic active suspension - ResearchGate, accessed June 5, 2026, https://www.researchgate.net/publication/403850098_Smith-predictor-based_variable-universe_fuzzy_position_control_for_hydraulic_active_suspension
30. Machine-Learning-Based Improved Smith Predictive Control for MIMO Processes - MDPI, accessed June 5, 2026, https://www.mdpi.com/2227-7390/10/19/3696
31. Output Tracking for Uncertain Time-Delay Systems via Robust Reinforcement Learning Control - IEEE Xplore, accessed June 5, 2026, https://ieeexplore.ieee.org/document/10662811/
32. How to deal with actions that complete in multiple steps (delayed reward) in reinforcement learning? - AI Stack Exchange, accessed June 5, 2026, https://ai.stackexchange.com/questions/48252/how-to-deal-with-actions-that-complete-in-multiple-steps-delayed-reward-in-rei
33. Delay-Robust Deep Reinforcement Learning for Ranging-Free Channel Access under Mobility in Underwater Acoustic Networks - arXiv, accessed June 5, 2026, https://arxiv.org/html/2605.06536v1

34. [2605.06536] Delay-Robust Deep Reinforcement Learning for Ranging-Free Channel Access under Mobility in Underwater Acoustic Networks - arXiv, accessed June 5, 2026, https://arxiv.org/abs/2605.06536
35. Reinforcement Learning-Based Control of a 4-Wheel Independent Steering Mobile Robot for Robust Path Tracking in Outdoor Environments - MDPI, accessed June 5, 2026, https://www.mdpi.com/1424-8220/26/6/1761
36. Enhancing Control Policy Smoothness by Aligning Actions with Predictions from Preceding States - arXiv, accessed June 5, 2026, https://arxiv.org/html/2601.18479v1
37. Learning Smooth Time-Varying Linear Policies with an Action Jacobian Penalty - arXiv, accessed June 5, 2026, https://arxiv.org/html/2602.18312v1
38. Enhancing Control Policy Smoothness by Aligning Actions with Predictions from Preceding States, accessed June 5, 2026, https://airlabkhu.github.io/ASAP/
39. Continual Domain Randomization - arXiv, accessed June 5, 2026, https://arxiv.org/html/2403.12193v1
40. DiAReL: Reinforcement Learning With Disturbance Awareness for Robust Sim2Real Policy Transfer in Robot Control - IEEE Xplore, accessed June 5, 2026, https://ieeexplore.ieee.org/iel8/87/4389040/11270848.pdf
41. ADR: Train Hard, Transfer Smart. Learned domain randomization selects... | by Dong-Keon Kim | Medium, accessed June 5, 2026, https://medium.com/@kdk199604/adr-train-hard-transfer-smart-bad19432c3b9
42. How to straight back a tractor trailer and how to straighten out ? | TruckersReport.com Trucking Forum, accessed June 5, 2026, https://www.thetruckersreport.com/truckingindustryforum/threads/how-to-straight-back-a-tractor-trailer-and-how-to-straighten-out.263751/
43. Domain randomization : r/reinforcementlearning - Reddit, accessed June 5, 2026, https://www.reddit.com/r/reinforcementlearning/comments/1lfccnt/domain_randomization/
44. Continual Domain Randomization - arXiv, accessed June 5, 2026, https://arxiv.org/pdf/2403.12193
45. comparison of modern controls and reinforcement learning for robust control of autonomously back - SciSpace, accessed June 5, 2026, https://scispace.com/pdf/comparison-of-modern-controls-and-reinforcement-learning-for-1zu7e2t8zm.pdf
46. Heterogeneous Self-Play for Realistic Highway Traffic Simulation - arXiv, accessed June 5, 2026, https://arxiv.org/html/2604.16406v1